Complete Tutorial on JPA (Java Persistence API) with Spring Boot

Java | Spring Boot | Spring Data JPA | Hibernate | MySQL

1. Introduction to JPA

What is JPA?

JPA (Java Persistence API), now officially called Jakarta Persistence, is a Java specification used to manage relational database data through Java objects.

JPA allows developers to perform database operations such as inserting, retrieving, updating, and deleting records without writing SQL queries for every operation.

JPA provides a standard approach to Object-Relational Mapping (ORM).

Definition: JPA is a specification that defines how Java objects are mapped to relational database tables and how persistent data is managed.

Example

Consider a database table named `students`.

id	name	email	course
101	Rahul	rahul@gmail.com	Java
102	Priya	priya@gmail.com	Python
103	Aman	aman@gmail.com	Spring Boot

In traditional JDBC, we write SQL queries:

```
String sql = "SELECT * FROM students";
```

Using Spring Data JPA, we can retrieve all student records with:

```
List<Student> students = studentRepository.findAll();
```

Spring Data JPA generates the required database operation using a JPA implementation such as Hibernate.

2. Why do we need JPA?

Before JPA, developers commonly used JDBC to interact with databases.

Problems with JDBC

- Developers manually write SQL queries.
- ResultSet data must be converted into Java objects.
- Repetitive database operations require additional code.
- Managing relationships between tables is more complex.
- Database-specific SQL can reduce portability.

Advantages of JPA

1. Reduces repetitive database code.
2. Supports object-relational mapping.
3. Supports relationships between entities.
4. Provides transaction management.
5. Supports JPQL (Jakarta Persistence Query Language).
6. Supports automatic primary key generation.
7. Supports persistence context and dirty checking.
8. Integrates with Spring Boot through Spring Data JPA.

3. JPA vs Hibernate vs Spring Data JPA

These three technologies are closely related but serve different purposes.

Technology	Description	Type
JPA	Defines persistence rules and APIs	Specification
Hibernate ORM	Implements JPA and provides additional ORM features	Framework
Spring Data JPA	Simplifies repository and data-access development	Spring module

Important: JPA itself is not a database framework implementation. Hibernate is one of the most commonly used implementations of JPA.

Architecture

Client (Browser / Postman)

Spring Boot REST Controller

Handles HTTP requests

Service Layer

Business logic

Spring Data JPA Repository

save(), findAll(), findById(), deleteById()

JPA API + Hibernate ORM

Maps objects and generates SQL

MySQL Database

4. Important JPA terminology

Term	Definition
Entity	Java class mapped to a database table
ORM	Technique of mapping Java objects to relational tables
Persistence	Storing and managing application data
EntityManager	JPA interface for managing entity lifecycle and persistence operations

Persistence Context	Environment that manages entity instances
Primary Key	Unique identifier for each record
JPQL	Object-oriented query language for JPA entities
Transaction	Logical unit of database operations
Repository	Interface used to access and manage database data
Hibernate	ORM framework implementing Jakarta Persistence

5. Important JPA annotations

Entity and table mapping annotations

Annotation	Description
`@Entity`	Marks a Java class as a persistent entity
`@Table`	Specifies the database table name
`@Id`	Identifies the primary key
`@GeneratedValue`	Configures primary key generation
`@Column`	Configures database column mapping
`@Transient`	Excludes a field from persistence
`@Enumerated`	Maps an enum to a database column
`@Lob`	Maps large object data
`@Temporal`	Specifies temporal precision for legacy Date/Calendar fields
`@Version`	Enables optimistic locking

Relationship annotations

Annotation	Description
`@OneToOne`	One entity is associated with one other entity
`@OneToMany`	One entity is associated with multiple entities
`@ManyToOne`	Multiple entities are associated with one entity
`@ManyToMany`	Multiple entities are associated with multiple entities
`@JoinColumn`	Specifies a foreign key column
`@JoinTable`	Specifies an association table

Spring Data JPA annotations

Annotation	Description
`@Repository`	Identifies a data-access component
`@Query`	Defines custom JPQL or native SQL queries
`@Modifying`	Marks modifying repository queries
`@Param`	Binds method parameters to named query parameters
`@EnableJpaRepositories`	Enables JPA repository scanning when explicit configuration is needed

Part 2: Spring Boot JPA Practical Example

We will create a Student Management REST API using Spring Boot, Spring Data JPA, Hibernate, and MySQL.

Project requirements

- Java 17 or later
- Spring Boot 3.x
- Maven
- MySQL Server
- VS Code, IntelliJ IDEA, or Eclipse
- Postman (optional)

6. Create the Spring Boot project

Open Spring Initializr and configure:

Setting	Value
Project	Maven
Language	Java
Spring Boot	Compatible Spring Boot 3.x version
Group	com.example
Artifact	student-jpa
Packaging	Jar
Java	17

Add dependencies:

- Spring Web
- Spring Data JPA
- MySQL Driver

Project structure

```
student-jpa/
│
├── src/main/java/com/example/studentjpa/
│   │
│   ├── StudentJpaApplication.java
│   │
│   ├── entity/
│   │   └── Student.java
│   │
│   ├── repository/
│   │   └── StudentRepository.java
│   │
│   ├── service/
│   │   └── StudentService.java
│   │
│   └── controller/
│       └── StudentController.java
│
├── src/main/resources/
│   └── application.properties
│
└── pom.xml
```

7. Maven configuration — pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.3.2</version>
        <relativePath/>
    </parent>

    <groupId>com.example</groupId>
    <artifactId>student-jpa</artifactId>
    <version>0.0.1-SNAPSHOT</version>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>

        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>

    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>

</project>
```

Spring Boot 3.3.2 is used here as a fixed teaching example. For new projects, choose a currently supported compatible Spring Boot version.

Dependency explanation

Spring Web: Used to create REST controllers and handle HTTP requests.

Spring Data JPA: Provides repository support and integrates with JPA implementations such as Hibernate.

MySQL Connector/J: JDBC driver used to connect the application to MySQL.

8. Create the database

Open MySQL Workbench and execute:

```
CREATE DATABASE studentdb;

USE studentdb;
```

We will allow Hibernate to create the `students` table automatically.

9. Configure application.properties

File: `src/main/resources/application.properties`

```
spring.application.name=student-jpa

server.port=8080

spring.datasource.url=jdbc:mysql://localhost:3306/studentdb
spring.datasource.username=root
spring.datasource.password=your_password

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

spring.jpa.properties.hibernate.format_sql=true

spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
```

Explanation

Property	Purpose
`spring.datasource.url`	Database connection URL
`spring.datasource.username`	Database username
`spring.datasource.password`	Database password
`spring.jpa.hibernate.ddl-auto`	Controls Hibernate schema management
`spring.jpa.show-sql`	Displays SQL statements
`hibernate.format_sql`	Formats Hibernate SQL logs
`spring.jpa.database-platform`	Specifies the Hibernate SQL dialect

Hibernate DDL options

Value	Description
`none`	Does not perform automatic schema management
`validate`	Checks whether the database schema matches entity mappings
`update`	Attempts to update the schema
`create`	Recreates the schema when the application starts
`create-drop`	Creates the schema at startup and drops it at shutdown

Note: Use `update` for classroom demonstrations. For production applications, prefer managed database migrations using Flyway or Liquibase.

10. Create the Student entity

File: `entity/Student.java`

```
package com.example.studentjpa.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Column(unique = true, nullable = false)
    private String email;

    private String course;

    public Student() {
    }

    public Student(String name, String email, String course) {
        this.name = name;
        this.email = email;
        this.course = course;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getCourse() {
        return course;
    }

    public void setCourse(String course) {
        this.course = course;
    }
}
```

Annotation explanation

`@Entity`

Marks the Java class as a JPA entity.

```
@Entity
public class Student {
}
```

`@Table`

Specifies the database table name.

```
@Table(name = "students")
```

`@Id`

Identifies the primary key.

```
@Id
private Long id;
```

`@GeneratedValue`

Automatically generates primary key values.

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

With MySQL, `IDENTITY` commonly uses an auto-increment column.

`@Column`

Configures column constraints.

```
@Column(nullable = false, length = 100)
private String name;
```

This indicates that the `name` column cannot contain `NULL` and has a maximum configured length of 100 characters.

Generated table

Hibernate can generate a table similar to:

```
CREATE TABLE students (
    id BIGINT NOT NULL AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    course VARCHAR(255),
    PRIMARY KEY (id)
);
```

The exact SQL depends on the Hibernate version and database dialect.

11. Create StudentRepository

File: `repository/StudentRepository.java`

```
package com.example.studentjpa.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.example.studentjpa.entity.Student;

public interface StudentRepository
        extends JpaRepository<Student, Long> {

}
```

Explanation

`JpaRepository<Student, Long>` means:

- `Student` is the entity type.
- `Long` is the primary key type.

Spring Data JPA automatically provides implementations of common repository operations.

Common repository methods

Method	Purpose
`save(entity)`	Insert or update
`findAll()`	Retrieve all records
`findById(id)`	Retrieve a record by ID
`existsById(id)`	Check whether an ID exists
`count()`	Count records
`deleteById(id)`	Delete a record by ID
`delete(entity)`	Delete an entity
`findAll(Sort)`	Retrieve sorted records
`findAll(Pageable)`	Retrieve paginated records

No implementation class needs to be written manually.

12. Create StudentService

File: `service/StudentService.java`

```
package com.example.studentjpa.service;

import java.util.List;
import java.util.Optional;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.example.studentjpa.entity.Student;
import com.example.studentjpa.repository.StudentRepository;

@Service
public class StudentService {

    private final StudentRepository repository;

    public StudentService(StudentRepository repository) {
        this.repository = repository;
```

```
    }

    // Insert Student
    public Student addStudent(Student student) {
        return repository.save(student);
    }

    // Retrieve All Students
    public List<Student> getAllStudents() {
        return repository.findAll();
    }

    // Search Student By ID
    public Optional<Student> getStudentById(Long id) {
        return repository.findById(id);
    }

    // Update Student
    @Transactional
    public Optional<Student> updateStudent(
            Long id, Student newStudent) {

        return repository.findById(id).map(student -> {

            student.setName(newStudent.getName());
            student.setEmail(newStudent.getEmail());
            student.setCourse(newStudent.getCourse());

            return repository.save(student);
        });
    }

    // Delete Student
    public boolean deleteStudent(Long id) {

        if (!repository.existsById(id)) {
            return false;
        }

        repository.deleteById(id);
        return true;
    }
}
```

Explanation

`@Service` marks this class as a service-layer component.

Constructor injection provides the `StudentRepository` dependency.

`@Transactional` ensures that the update operation executes within a transaction. The loaded entity is managed, and changes can be synchronized automatically by Hibernate through dirty checking. The explicit `save()` in this example is included for clarity, but is not required for an already managed entity.

13. Create StudentController

File: `controller/StudentController.java`

```
package com.example.studentjpa.controller;

import java.util.List;
```

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import com.example.studentjpa.entity.Student;
import com.example.studentjpa.service.StudentService;

@RestController
@RequestMapping("/students")
public class StudentController {

    private final StudentService service;

    public StudentController(StudentService service) {
        this.service = service;
    }

    // POST - Insert Student
    @PostMapping
    public ResponseEntity<Student> addStudent(
            @RequestBody Student student) {

        Student saved = service.addStudent(student);

        return ResponseEntity
                .status(HttpStatus.CREATED)
                .body(saved);
    }

    // GET - All Students
    @GetMapping
    public List<Student> getAllStudents() {
        return service.getAllStudents();
    }

    // GET - Student By ID
    @GetMapping("/{id}")
    public ResponseEntity<Student> getStudentById(
            @PathVariable Long id) {

        return service.getStudentById(id)
                .map(ResponseEntity::ok)
                .orElseGet(() ->
                    ResponseEntity.notFound().build());
    }

    // PUT - Update Student
    @PutMapping("/{id}")
    public ResponseEntity<Student> updateStudent(
            @PathVariable Long id,
            @RequestBody Student student) {

        return service.updateStudent(id, student)
                .map(ResponseEntity::ok)
                .orElseGet(() ->
                    ResponseEntity.notFound().build());
    }

    // DELETE - Delete Student
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteStudent(
            @PathVariable Long id) {

        boolean deleted = service.deleteStudent(id);

        if (deleted) {
            return ResponseEntity.noContent().build();
```

```
        }

        return ResponseEntity.notFound().build();
    }
}
```

14. Main application class

File: `StudentJpaApplication.java`

```
package com.example.studentjpa;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class StudentJpaApplication {

    public static void main(String[] args) {
        SpringApplication.run(
            StudentJpaApplication.class, args);
    }
}
```

15. Run the application

Open the terminal in the project directory:

```
mvn spring-boot:run
```

Once the application starts successfully, Hibernate creates or updates the table according to the configured DDL mode.

Test the REST APIs

POST — Insert Student

201 Created

Endpoint: `http://localhost:8080/students`

Request body:

```
{
  "name": "Rahul",
  "email": "rahul@gmail.com",
  "course": "Java"
}
```

Example response:

```
{
  "id": 1,
  "name": "Rahul",
  "email": "rahul@gmail.com",
  "course": "Java"
}
```

GET — Retrieve All Students

200 OK

Endpoint: `http://localhost:8080/students`

```
[
  {
    "id": 1,
    "name": "Rahul",
    "email": "rahul@gmail.com",
    "course": "Java"
  }
]
```

GET — Search By ID

200 OK

Endpoint: `http://localhost:8080/students/1`

Returns the matching student or `404 Not Found`.

PUT — Update Student

200 OK

Endpoint: `http://localhost:8080/students/1`

```
{
  "name": "Rahul Sharma",
  "email": "rahul@gmail.com",
  "course": "Spring Boot"
}
```

DELETE — Delete Student

204 No Content

Endpoint: `http://localhost:8080/students/1`

Deletes the record and returns an empty response body.

The examples assume a fresh database and valid input. Duplicate emails or invalid data may produce database errors unless additional validation and exception handling are implemented.

Part 3: JPA Advanced Concepts

16. Primary key generation strategies

JPA supports several strategies for generating primary keys.

Strategy	Description
`AUTO`	Provider chooses an appropriate generation strategy
`IDENTITY`	Uses database identity or auto-increment support
`SEQUENCE`	Uses a database sequence
`TABLE`	Uses a separate table for ID generation

`UUID`	Generates UUID identifiers in Jakarta Persistence 3.1+

Example: IDENTITY

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

Example: SEQUENCE

```
@Id
@GeneratedValue(
    strategy = GenerationType.SEQUENCE,
    generator = "student_seq"
)
@SequenceGenerator(
    name = "student_seq",
    sequenceName = "student_sequence",
    allocationSize = 1
)
private Long id;
```

Sequence generation is appropriate for databases with sequence support, such as PostgreSQL and Oracle.

17. JPA entity lifecycle

JPA entities pass through different lifecycle states.

Transient

New object, not managed

Managed

Tracked by persistence context

persist / find / merge

Detached

Previously managed, no longer tracked

Removed

Scheduled for deletion

Illustrative states; transitions depend on EntityManager operations.

Transient state

An entity object is created but is not managed by JPA.

```
Student student = new Student(
    "Aman", "aman@gmail.com", "Java");
```

Managed state

The entity is associated with a persistence context.

```
Student student = entityManager.find(Student.class, 1L);
```

Detached state

An entity was managed but is no longer associated with an active persistence context.

```
entityManager.detach(student);
```

Removed state

The entity is marked for deletion.

```
entityManager.remove(student);
```

18. EntityManager in JPA

`EntityManager` is a core JPA interface responsible for managing entity persistence and lifecycle operations.

Common EntityManager methods

Method	Description
`persist()`	Makes a new entity managed
`find()`	Finds an entity by primary key
`merge()`	Copies detached entity state into a managed entity
`remove()`	Marks an entity for deletion
`flush()`	Synchronizes pending changes with the database
`detach()`	Detaches an entity
`clear()`	Detaches all managed entities
`refresh()`	Reloads entity state from the database

Example: EntityManager CRUD

```
@Service
public class StudentEntityManagerService {

    @PersistenceContext
    private EntityManager entityManager;

    @Transactional
    public void insert(Student student) {
        entityManager.persist(student);
    }

    @Transactional(readOnly = true)
    public Student search(Long id) {
        return entityManager.find(Student.class, id);
    }

    @Transactional
    public void delete(Long id) {
        Student student =
            entityManager.find(Student.class, id);

        if (student != null) {
            entityManager.remove(student);
```

```
        }
    }
}
```

Required imports:

```
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
```

EntityManager vs JpaRepository

EntityManager	JpaRepository
Low-level JPA API	Higher-level Spring Data abstraction
Requires explicit persistence operations	Provides ready-made CRUD methods
Offers fine-grained persistence control	Reduces boilerplate code
Uses `persist()`, `merge()`, `remove()`	Uses `save()`, `findAll()`, `delete()`

19. Custom queries in JPA

Spring Data JPA supports different approaches to querying records.

A. Derived query methods

Add the following methods to `StudentRepository`:

```
List<Student> findByName(String name);

List<Student> findByCourse(String course);

List<Student> findByNameContaining(String name);

List<Student> findByCourseOrderByNameAsc(String course);

long countByCourse(String course);

boolean existsByEmail(String email);
```

Spring Data JPA derives queries from the method names.

B. JPQL queries

JPQL uses entity names and Java property names rather than database table and column names.

```
@Query("SELECT s FROM Student s WHERE s.course = :course")
List<Student> searchByCourse(
    @Param("course") String course);
```

C. Native SQL queries

Native queries use the database's SQL syntax.

```
@Query(
    value = "SELECT * FROM students WHERE course = :course",
    nativeQuery = true
)
```

```
List<Student> searchNative(
    @Param("course") String course);
```

JPQL vs SQL

JPQL	SQL
Queries entities	Queries database tables
Uses Java property names	Uses database column names
Generally database independent	Can contain database-specific syntax
Returns entity objects or projections	Returns database result data

20. Pagination and sorting

Pagination divides large datasets into smaller pages.

Pagination example

```
Page<Student> findAll(Pageable pageable);
```

`JpaRepository` already inherits this method, so you do not need to declare it.

Service method:

```
public Page<Student> getStudents(int page, int size) {

    Pageable pageable = PageRequest.of(page, size);

    return repository.findAll(pageable);
}
```

Required imports:

```
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.PageRequest;
```

Sorting example

```
List<Student> students = repository.findAll(
    Sort.by("name").ascending()
);
```

Required import:

```
import org.springframework.data.domain.Sort;
```

Controller example

```
@GetMapping("/page")
public Page<Student> getStudentsPage(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "5") int size) {

    return service.getStudents(page, size);
}
```

Example request:

```
GET http://localhost:8080/students/page?page=0&size=5
```

Page numbering starts from zero.

Part 4: JPA Relationships

21. Types of relationships

Relational databases often contain multiple tables connected through primary keys and foreign keys.

JPA provides annotations to represent these relationships using Java objects.

One-to-One

Person

Passport

One-to-Many

Department

Employees

Many-to-One

Employees

Department

Many-to-Many

Students

Courses

22. One-to-One relationship

Definition: One record in one table is associated with one record in another table.

Example: One student has one identity card.

Student entity

```
@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "identity_card_id")
```

```
    private IdentityCard identityCard;

    // Constructors, getters and setters
}
```

IdentityCard entity

```
@Entity
public class IdentityCard {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String cardNumber;

    // Constructors, getters and setters
}
```

This creates a foreign key relationship using `identity_card_id` in the Student table.

23. One-to-Many and Many-to-One relationship

Definition: One department can have multiple employees, but each employee belongs to one department.

Department entity

```
@Entity
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String departmentName;

    @OneToMany(mappedBy = "department")
    private List<Employee> employees = new ArrayList<>();

    // Constructors, getters and setters
}
```

Employee entity

```
@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String employeeName;

    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;

    // Constructors, getters and setters
}
```

Database tables

Department

id	department_name
1	IT
2	HR

Employee

id	employee_name	department_id
101	Rahul	1
102	Priya	1
103	Aman	2

The `department_id` column is a foreign key.

The `mappedBy = "department"` attribute indicates that the `department` field in the Employee entity owns the relationship.

24. Many-to-Many relationship

Definition: Multiple students can enroll in multiple courses.

Student entity

```
@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @ManyToMany
    @JoinTable(
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private Set<Course> courses = new HashSet<>();

    // Constructors, getters and setters
}
```

Course entity

```
@Entity
public class Course {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String courseName;

    @ManyToMany(mappedBy = "courses")
    private Set<Student> students = new HashSet<>();
```

```
    // Constructors, getters and setters
}
```

JPA uses an association table named `student_course` to store the relationship.

Note: These relationship examples are separate teaching examples. Add the appropriate imports, constructors, getters, and setters before running them. When exposing bidirectional entities through REST APIs, use DTOs to prevent recursive JSON serialization.

25. Cascade types in JPA

Cascade operations allow an operation on one entity to be propagated to associated entities.

Cascade type	Description
`PERSIST`	Propagates persist operations
`MERGE`	Propagates merge operations
`REMOVE`	Propagates remove operations
`REFRESH`	Propagates refresh operations
`DETACH`	Propagates detach operations
`ALL`	Includes all the above operations

Example:

```
@OneToOne(cascade = CascadeType.ALL)
private IdentityCard identityCard;
```

Deleting a Student can also delete its associated IdentityCard when `REMOVE` cascading is enabled.

Be careful using `CascadeType.REMOVE` or `ALL` on shared relationships, particularly `@ManyToMany`.

26. Fetch types — EAGER vs LAZY

Fetch type controls when related entities are loaded.

EAGER fetching

```
@ManyToOne(fetch = FetchType.EAGER)
private Department department;
```

The related Department is fetched as part of loading the Employee, though Hibernate may use a join or separate SQL queries.

LAZY fetching

```
@ManyToOne(fetch = FetchType.LAZY)
private Department department;
```

The Department is normally loaded when the relationship is accessed, assuming the required persistence context is available.

EAGER	LAZY
Loads associated data immediately	Defers associated data loading

Can retrieve unnecessary data	Can reduce unnecessary loading
May cause excessive queries	Can cause lazy initialization errors if accessed outside an active persistence context

JPA defaults are generally EAGER for `@ManyToOne` and `@OneToOne`, and LAZY for `@OneToMany` and `@ManyToMany`.

Part 5: Transactions and Performance

27. Transaction management

A transaction is a group of database operations treated as one logical unit.

JPA commonly uses Spring's `@Transactional` annotation for transaction management.

Example: Transfer fees between accounts

```
@Service
public class AccountService {

    private final AccountRepository repository;

    public AccountService(AccountRepository repository) {
        this.repository = repository;
    }

    @Transactional
    public void transfer(Long fromId, Long toId, double amount) {

        Account from = repository.findById(fromId)
            .orElseThrow();

        Account to = repository.findById(toId)
            .orElseThrow();

        from.setBalance(from.getBalance() - amount);
        to.setBalance(to.getBalance() + amount);

        repository.save(from);
        repository.save(to);
    }
}
```

If an unchecked exception occurs, Spring normally rolls back the transaction.

For real financial applications, use `BigDecimal`, validate balances and transfer amounts, and address concurrency.

28. Dirty checking

Dirty checking is a Hibernate mechanism that automatically detects changes made to managed entities.

```
@Transactional
public void updateCourse(Long id) {

    Student student = repository.findById(id)
        .orElseThrow();
```

```
    student.setCourse("Advanced Java");
}
```

When the transaction commits, Hibernate normally detects the change and issues an SQL `UPDATE`.

There is no need to explicitly call `save()` for this managed entity.

29. First-level cache

The persistence context acts as a first-level cache.

```
@Transactional
public void testCache() {

    Student s1 = entityManager.find(Student.class, 1L);

    Student s2 = entityManager.find(Student.class, 1L);
}
```

For an entity already loaded in the same persistence context, the second `find()` generally does not require another SQL query.

This cache is scoped to the persistence context and is enabled by default.

30. N+1 query problem

The N+1 query problem occurs when retrieving a collection of entities causes additional queries to load their relationships.

Example:

- One query retrieves 10 departments.
- Ten additional queries retrieve employees for each department.

Total: 11 queries.

Solution using EntityGraph

```
public interface DepartmentRepository
        extends JpaRepository<Department, Long> {

    @EntityGraph(attributePaths = "employees")
    @Query("SELECT d FROM Department d")
    List<Department> findAllWithEmployees();
}
```

This requests that employees be fetched with departments, often avoiding the additional per-department queries.

31. Optimistic locking

Optimistic locking prevents silently overwriting concurrent updates.

```
@Entity
public class Student {

    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @Version
    private Long version;
}
```

When two transactions attempt conflicting updates, Hibernate can detect that the entity version has changed and raise an optimistic locking exception.

Part 6: Common Errors in JPA

Error	Possible cause	Solution
`Not a managed type`	Missing `@Entity` or entity scanning issue	Add `@Entity` and verify package scanning
`No identifier specified`	Missing primary key	Add `@Id`
`Unable to determine Dialect`	Database connection/configuration issue	Check datasource and JDBC driver
`Table doesn't exist`	Schema not created	Check database and DDL settings
`LazyInitializationException`	Lazy relationship accessed after persistence context closes	Use a transactional service method or DTO projection
`DataIntegrityViolationException`	Constraint violation	Check unique keys, foreign keys, and required fields
`detached entity passed to persist`	Attempting to persist a detached entity	Review entity state and relationship cascades
`package jakarta.persistence does not exist`	Missing JPA dependency or incompatible setup	Add Spring Data JPA starter and reload Maven

Important: javax.persistence vs jakarta.persistence

Spring Boot 3 uses Jakarta Persistence packages:

```
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.GeneratedValue;
```

Older applications based on Spring Boot 2 often use:

```
import javax.persistence.Entity;
```

Do not mix the old `javax.persistence` API with a Spring Boot 3 Jakarta Persistence application.

Part 7: Practical Exercises for Students

32. Lab exercises

Complete these exercises using the Student Management application developed above.

Student practice checklist

0 of 15 completed

- [ ] 1. Create a Student entity with id, name, email, course, age, and city.
- [ ] 2. Insert five students using POST requests.
- [ ] 3. Retrieve all students using JpaRepository.
- [ ] 4. Search a student by primary key.
- [ ] 5. Update a student's course using PUT.
- [ ] 6. Delete a student by ID.
- [ ] 7. Create a repository method findByCity(String city).
- [ ] 8. Retrieve students whose names contain a given keyword.
- [ ] 9. Sort students by name in ascending order.
- [ ] 10. Implement pagination with five records per page.
- [ ] 11. Create Department and Employee entities with a Many-to-One relationship.
- [ ] 12. Write a JPQL query to retrieve students by course.
- [ ] 13. Write a native SQL query to retrieve students by city.
- [ ] 14. Implement transaction management using @Transactional.
- [ ] 15. Add @Version and demonstrate optimistic locking.

Reset checklist

33. Interview and viva questions

1. What is JPA, and why is it used?
2. What is the difference between JPA and Hibernate?
3. What is Spring Data JPA?
4. What is an entity?
5. What is the purpose of `@Id`?
6. What is the difference between `persist()` and `merge()`?
7. What is the difference between `save()` and `saveAndFlush()`?
8. What is the persistence context?
9. What is dirty checking?
10. What is the difference between EAGER and LAZY fetching?
11. What are cascade types?
12. What is the purpose of `mappedBy`?
13. What is the difference between JPQL and native SQL?
14. What is the N+1 query problem?
15. What is optimistic locking?
16. What is the role of `@Transactional`?
17. What is the difference between `CrudRepository` and `JpaRepository`?
18. How do pagination and sorting work?
19. What is the difference between `@OneToMany` and `@ManyToOne`?
20. Why should DTOs be used in REST applications?

34. Suggested teaching plan

This tutorial can be divided into approximately six hours of classroom instruction.

Session	Topics	Duration

1	JPA introduction, ORM, Hibernate, architecture	45 min
2	Spring Boot project setup, database configuration, entities	60 min
3	Repository, service, controller, CRUD API execution	90 min
4	JPQL, native queries, pagination and sorting	60 min
5	JPA relationships, cascading, fetching	60 min
6	EntityManager, transactions, dirty checking, revision	45 min

35. Summary

JPA is a fundamental technology for database-driven Java applications. It provides a standard API for mapping Java objects to relational database records.

The most important concepts students should understand are:

- JPA: The persistence specification.
- Hibernate: The ORM implementation.
- Spring Data JPA: Simplifies repository operations.
- Entity: Represents persistent data.
- JpaRepository: Provides ready-made database methods.
- EntityManager: Manages entity persistence and lifecycle.
- JPQL: Queries entities using Java-oriented syntax.
- Relationships: Connect entities using one-to-one, one-to-many, many-to-one, and many-to-many mappings.
- Transactions: Maintain consistency across database operations.
- Persistence context: Tracks managed entities and enables dirty checking.

Official references

For additional reading and classroom reference:

- Spring Data JPA Reference Documentation
- Hibernate ORM Documentation
- Jakarta Persistence Specification

The recommended learning sequence is Entity → Repository → CRUD → JPQL → Relationships → EntityManager → Transactions → Performance, with practical execution after each topic.

Convert to PDF
Is Hibernate in this tutorial

Yes, Hibernate is included in this JPA tutorial. It is covered as the implementation of JPA, with practical examples.

The tutorial includes Hibernate in these sections:

1. Section 3 — JPA vs Hibernate vs Spring Data JPA: Explains the differences.
2. Section 7 — Maven configuration: Includes the Spring Data JPA dependency, which brings Hibernate as the default JPA provider in a standard Spring Boot setup.
3. Section 9 — `application.properties`: Covers Hibernate configuration, including `spring.jpa.hibernate.ddl-auto` and SQL logging.
4. Section 10 — Entity mapping: Shows how Hibernate maps Java entities to database tables.
5. Section 15 — Application execution: Explains automatic table creation through Hibernate.

6. Sections 28–30 — Advanced concepts: Covers dirty checking, first-level caching, and the N+1 query problem.

However, the tutorial does not yet cover Hibernate as a separate, complete topic. It focuses mainly on using Hibernate through JPA and Spring Data JPA.

If you want to teach students Hibernate thoroughly, additional topics should include:

- Hibernate architecture and configuration
- `SessionFactory` and `Session`
- Hibernate CRUD operations
- HQL (Hibernate Query Language)
- Hibernate annotations
- First-level and second-level caching
- Lazy and eager loading
- Hibernate inheritance mapping
- Hibernate transaction management

Recommended teaching sequence: JDBC → Hibernate fundamentals → JPA → Spring Data JPA.

This helps students understand how database access evolved from manual SQL to ORM and then to repository-based development.

Convert to PDF
Create a Complete and detailed tutorial of Hibernate with example

Complete Hibernate Tutorial with Practical Java Examples

For beginners and B.Tech students | Java | Maven | Hibernate ORM | MySQL | CRUD | HQL | Relationships

This tutorial explains Hibernate from the basics to advanced concepts, using practical Java code that students can execute step by step.

1. What is Hibernate?

Hibernate is an Object-Relational Mapping (ORM) framework for Java. It helps developers store, retrieve, update, and delete database records using Java objects instead of manually converting every database row into a Java object.

Hibernate maps Java classes to database tables and Java fields to table columns. It generates SQL statements to perform database operations.

For example, suppose we have a Java class:

```
public class Student {
    private int id;
    private String name;
    private String course;
}
```

And a database table:

id	name	course
1	Rahul	Java
2	Priya	Python
3	Aman	Spring Boot

Hibernate can map a `Student` object to a row in the `students` table.

Instead of manually writing an `INSERT` statement and mapping each column, we can write:

```
session.persist(student);
```

Hibernate generates the necessary SQL when the changes are synchronized with the database.

2. Why do we need Hibernate?

Before ORM frameworks, Java developers commonly used JDBC.

Using JDBC

```
String sql =
    "INSERT INTO students(name, course) VALUES (?, ?)";

PreparedStatement ps = connection.prepareStatement(sql);
ps.setString(1, "Rahul");
ps.setString(2, "Java");

ps.executeUpdate();
```

With JDBC, developers must manage connections, SQL statements, parameters, result sets, and object mapping.

Using Hibernate

```
Student student = new Student();
student.setName("Rahul");
student.setCourse("Java");

session.persist(student);
```

Hibernate manages the object-to-table mapping and generates the required SQL.

Advantages of Hibernate

- Reduces repetitive JDBC code.
- Maps Java objects to database tables.
- Supports CRUD operations.
- Supports relationships between entities.
- Provides HQL and criteria-based queries.
- Supports transactions and caching.
- Supports lazy loading and dirty checking.
- Works with multiple relational database systems.
- Integrates with Spring and Spring Boot.

Important: Hibernate does not eliminate SQL completely. Developers may still use SQL for database administration, complex queries, performance tuning, and reporting.

3. JPA vs Hibernate vs Spring Data JPA

These technologies are related, but they are not the same.

Feature	JPA / Jakarta Persistence	Hibernate ORM	Spring Data JPA
Type	Specification	ORM implementation	Spring data-access module
Main purpose	Defines persistence APIs and rules	Implements ORM and persistence APIs	Simplifies repository-based database access
Entity annotations	Defines standard annotations	Implements them and adds extensions	Uses them through its JPA integration
CRUD	Standard APIs through `EntityManager`	Provides `Session` and JPA APIs	Provides repository methods
Example	`@Entity`, `@Id`	`Session`, `SessionFactory`	`JpaRepository`

A standard Spring Boot application using Spring Data JPA normally uses Hibernate as its default JPA provider.

In this tutorial, we will first use Hibernate directly, then explain how it relates to JPA and Spring Boot.

4. Hibernate architecture

Java Application

Student objects and business logic

Hibernate Session / JPA EntityManager

Persistence operations and entity management

Persistence Context and ORM Engine

Entity tracking, SQL generation and caching

JDBC Driver

Communicates with the database

MySQL Database

Tables, rows, constraints and indexes

Main components

Component	Definition
`Configuration`	Legacy Hibernate API used to load configuration and mappings
`SessionFactory`	Creates Hibernate sessions; typically shared across the application
`Session`	Represents a unit of interaction with the database and manages entities
`Transaction`	Groups database operations into a transactional unit
`Query`	Executes HQL, JPQL or SQL queries, depending on the API
`Persistent object`	Java object whose state Hibernate manages
`JDBC Driver`	Connects Java to the relational database

A `SessionFactory` is generally created once per database configuration. A `Session` is created for a unit of work and should not be shared concurrently across threads.

5. Create a Hibernate project using Maven

We will create a standalone Java application called `hibernate-demo` that connects to MySQL directly, without Spring Boot.

Step 1: Create the database

Open MySQL Workbench and execute:

```
CREATE DATABASE hibernate_db;

USE hibernate_db;
```

Hibernate will create the `students` table when the application starts.

Step 2: Create the project structure

```
hibernate-demo/
│
├── pom.xml
│
└── src/
    └── main/
        ├── java/
        │   └── com/example/
        │       ├── Student.java
        │       ├── HibernateUtil.java
        │       └── Main.java
        │
        └── resources/
            └── hibernate.cfg.xml
```

Step 3: Configure pom.xml

The following example uses Hibernate ORM 6.6.4.Final, which supports Java 17 and Jakarta Persistence 3.1.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>hibernate-demo</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.release>17</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>

        <!-- Hibernate ORM -->
        <dependency>
```

```
            <groupId>org.hibernate.orm</groupId>
            <artifactId>hibernate-core</artifactId>
            <version>6.6.4.Final</version>
        </dependency>

        <!-- MySQL JDBC Driver -->
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <version>8.4.0</version>
        </dependency>

        <!-- Logging implementation -->
        <dependency>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-simple</artifactId>
            <version>2.0.16</version>
            <scope>runtime</scope>
        </dependency>

    </dependencies>

</project>
```

Dependency explanation

- `hibernate-core`: Provides Hibernate's ORM functionality.
- `mysql-connector-j`: Connects Java to MySQL using JDBC.
- `slf4j-simple`: Provides a basic logging implementation.
- `maven.compiler.release`: Compiles the application for Java 17.

These versions are fixed for this teaching example; use a currently maintained compatible release set for a new production project.

Step 4: Configure Hibernate

Create `src/main/resources/hibernate.cfg.xml`.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "https://hibernate.org/dtd/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
    <session-factory>

        <!-- Database connection -->
        <property name="hibernate.connection.driver_class">
            com.mysql.cj.jdbc.Driver
        </property>

        <property name="hibernate.connection.url">
            jdbc:mysql://localhost:3306/hibernate_db
        </property>

        <property name="hibernate.connection.username">
            root
        </property>

        <property name="hibernate.connection.password">
            YOUR_MYSQL_PASSWORD
        </property>
```

```
        <!-- Hibernate SQL dialect -->
        <property name="hibernate.dialect">
            org.hibernate.dialect.MySQLDialect
        </property>

        <!-- Display SQL -->
        <property name="hibernate.show_sql">true</property>
        <property name="hibernate.format_sql">true</property>

        <!-- Create or update tables -->
        <property name="hibernate.hbm2ddl.auto">update</property>

        <!-- Register the entity -->
        <mapping class="com.example.Student"/>

    </session-factory>
</hibernate-configuration>
```

Replace `YOUR_MYSQL_PASSWORD` with your local MySQL password.

Important configuration properties

Property	Purpose
`hibernate.connection.url`	Database URL
`hibernate.connection.username`	Database user
`hibernate.connection.password`	Database password
`hibernate.dialect`	Determines database-specific SQL behavior
`hibernate.show_sql`	Displays SQL statements
`hibernate.format_sql`	Formats SQL output
`hibernate.hbm2ddl.auto`	Controls automatic schema management
`mapping class`	Registers the entity class

For classroom practice, `update` is convenient. For production, avoid relying on automatic schema changes; use controlled database migrations.

6. Create your first Hibernate entity

Create `src/main/java/com/example/Student.java`.

```
package com.example;

import jakarta.persistence.Entity;
import jakarta.persistence.Table;
import jakarta.persistence.Id;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Column;

@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;
```

```
    @Column(unique = true, nullable = false)
    private String email;

    private String course;

    // Required no-argument constructor
    public Student() {
    }

    public Student(String name, String email, String course) {
        this.name = name;
        this.email = email;
        this.course = course;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getCourse() {
        return course;
    }

    public void setCourse(String course) {
        this.course = course;
    }

    @Override
    public String toString() {
        return "Student{id=" + id
                + ", name='" + name + '\''
                + ", email='" + email + '\''
                + ", course='" + course + '\''
                + '}';
    }
}
```

Understand the annotations

Annotation	Meaning
`@Entity`	Tells Hibernate that this class represents a persistent entity
`@Table(name = "students")`	Maps the entity to the `students` table
`@Id`	Marks the primary key

`@GeneratedValue`	Configures automatic ID generation
`GenerationType.IDENTITY`	Uses the database's identity or auto-increment feature
`@Column`	Configures the mapped column
`nullable = false`	Makes the column non-nullable
`unique = true`	Specifies a uniqueness constraint

Why do we need a no-argument constructor?

Hibernate needs a no-argument constructor to instantiate entity objects. A public or protected no-argument constructor is suitable for a typical entity.

What does Hibernate do with this class?

Hibernate reads the entity mapping and can create a database table similar to:

```
CREATE TABLE students (
    id BIGINT NOT NULL AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    course VARCHAR(255),
    PRIMARY KEY (id)
);
```

The actual generated SQL may differ depending on Hibernate's version and the database.

7. Create HibernateUtil

Create `src/main/java/com/example/HibernateUtil.java`.

```
package com.example;

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class HibernateUtil {

    private static final SessionFactory SESSION_FACTORY =
            buildSessionFactory();

    private static SessionFactory buildSessionFactory() {
        try {
            return new Configuration()
                    .configure("hibernate.cfg.xml")
                    .buildSessionFactory();
        } catch (Throwable ex) {
            throw new ExceptionInInitializerError(ex);
        }
    }

    public static SessionFactory getSessionFactory() {
        return SESSION_FACTORY;
    }

    public static void shutdown() {
        SESSION_FACTORY.close();
    }
}
```

Explanation

`Configuration` loads Hibernate configuration and entity mappings.

`configure("hibernate.cfg.xml")` reads the XML configuration file.

`buildSessionFactory()` creates the `SessionFactory` from the configuration.

`getSessionFactory()` allows other classes to obtain the shared factory.

`shutdown()` closes the factory and releases associated resources.

In this example, the factory is initialized once and reused. In a real application, handle startup failures and application shutdown carefully.

8. Understand Session, SessionFactory and Transaction

These are core concepts in direct Hibernate programming.

SessionFactory

Creates `Session` instances. It is relatively expensive to build and should generally be shared.

```
SessionFactory factory = HibernateUtil.getSessionFactory();
```

Session

Provides methods to interact with the persistence context and database.

```
Session session = factory.openSession();
```

A session is generally short-lived and must be closed when finished.

Transaction

Groups database changes into a single unit of work.

```
Transaction tx = session.beginTransaction();
```

Commit the changes:

```
tx.commit();
```

Roll back on failure:

```
tx.rollback();
```

A good pattern is to use `try`/`catch`/`finally` or try-with-resources to close sessions reliably.

9. Insert a record using Hibernate

Create `src/main/java/com/example/Main.java`.

```
package com.example;

import org.hibernate.Session;
import org.hibernate.Transaction;
```

```
public class Main {

    public static void main(String[] args) {

        Transaction transaction = null;

        try (Session session =
                HibernateUtil.getSessionFactory().openSession()) {

            transaction = session.beginTransaction();

            Student student = new Student(
                    "Rahul",
                    "rahul@gmail.com",
                    "Java"
            );

            session.persist(student);

            transaction.commit();

            System.out.println("Student inserted successfully.");
            System.out.println(student);

        } catch (Exception e) {

            if (transaction != null
                    && transaction.isActive()) {
                transaction.rollback();
            }

            e.printStackTrace();

        } finally {
            HibernateUtil.shutdown();
        }
    }
}
```

Step-by-step execution

1. Hibernate creates or retrieves the shared `SessionFactory`.
2. The application opens a session.
3. A database transaction begins.
4. A new Student object is created.
5. `session.persist(student)` makes the object managed and schedules its insertion.
6. Hibernate synchronizes pending changes with the database, usually at flush or transaction commit.
7. The transaction commits.
8. The session closes and resources are released.

Expected database record

```
SELECT * FROM students;
```

id	name	email	course
1	Rahul	rahul@gmail.com	Java

The generated ID depends on the existing database records.

Important note about `persist()`

Calling `persist()` does not necessarily execute the SQL immediately. Hibernate may delay the insert until it flushes the persistence context or commits the transaction.

10. Retrieve records using Hibernate

Hibernate provides the `find()` method to retrieve an entity using its primary key.

Example: Search student by ID

```
package com.example;

import org.hibernate.Session;

public class SearchStudent {

    public static void main(String[] args) {

        try (Session session =
                HibernateUtil.getSessionFactory().openSession()) {

            Student student = session.find(Student.class, 1L);

            if (student != null) {
                System.out.println(student);
            } else {
                System.out.println("Student not found.");
            }

        } finally {
            HibernateUtil.shutdown();
        }
    }
}
```

Explanation

```
session.find(Student.class, 1L);
```

- `Student.class` specifies the entity type.
- `1L` is the primary key, represented as a `Long`.
- Hibernate retrieves the matching entity.
- If the record does not exist, `find()` returns `null`.

Example output:

```
Student{id=1, name='Rahul', email='rahul@gmail.com', course='Java'}
```

11. Retrieve all records using HQL

HQL (Hibernate Query Language) is an object-oriented query language that uses entity names and Java property names instead of database table and column names.

Create `DisplayStudents.java`:

```
package com.example;

import java.util.List;
import org.hibernate.Session;
```

```
public class DisplayStudents {

    public static void main(String[] args) {

        try (Session session =
                HibernateUtil.getSessionFactory().openSession()) {

            List<Student> students = session
                    .createQuery("FROM Student", Student.class)
                    .getResultList();

            for (Student student : students) {
                System.out.println(student);
            }

        } finally {
            HibernateUtil.shutdown();
        }
    }
}
```

Explanation

`session.createQuery("FROM Student", Student.class)`

This creates a typed HQL query that returns Student objects.

`.getResultList();`

Executes the query and returns the results as a list.

Hibernate generates SQL similar to:

```
SELECT id, name, email, course
FROM students;
```

The exact SQL depends on the database and Hibernate configuration.

HQL vs SQL

HQL	SQL
Uses entity names	Uses table names
Uses Java property names	Uses database column names
`FROM Student`	`SELECT * FROM students`
Can return entity objects	Returns database result data
Usually database-independent	Can use database-specific features

12. Update a record using Hibernate

Hibernate automatically tracks changes to managed entities. This feature is called dirty checking.

Create `UpdateStudent.java`:

```
package com.example;

import org.hibernate.Session;
import org.hibernate.Transaction;
```

```
public class UpdateStudent {

    public static void main(String[] args) {

        Transaction transaction = null;

        try (Session session =
                HibernateUtil.getSessionFactory().openSession()) {

            transaction = session.beginTransaction();

            Student student = session.find(Student.class, 1L);

            if (student != null) {
                student.setCourse("Spring Boot");

                transaction.commit();

                System.out.println("Student updated.");
            } else {
                System.out.println("Student not found.");
                transaction.commit();
            }

        } catch (Exception e) {

            if (transaction != null && transaction.isActive()) {
                transaction.rollback();
            }

            e.printStackTrace();

        } finally {
            HibernateUtil.shutdown();
        }
    }
}
```

How does it work?

1. Hibernate loads the student.
2. The entity becomes managed by the session.
3. The course property changes.
4. Hibernate detects the modification.
5. Hibernate issues an SQL `UPDATE` when changes are flushed.
6. The transaction commits the change.

No explicit `session.update()` or `session.merge()` is required for an entity already managed by the session.

13. Delete a record using Hibernate

Create `DeleteStudent.java`:

```
package com.example;

import org.hibernate.Session;
import org.hibernate.Transaction;

public class DeleteStudent {

    public static void main(String[] args) {
```

```
        Transaction transaction = null;

        try (Session session =
                HibernateUtil.getSessionFactory().openSession()) {

            transaction = session.beginTransaction();

            Student student = session.find(Student.class, 1L);

            if (student != null) {
                session.remove(student);
                System.out.println("Student marked for deletion.");
            } else {
                System.out.println("Student not found.");
            }

            transaction.commit();

        } catch (Exception e) {

            if (transaction != null && transaction.isActive()) {
                transaction.rollback();
            }

            e.printStackTrace();

        } finally {
            HibernateUtil.shutdown();
        }
    }
}
```

Explanation

`session.remove(student);`

Marks the managed entity for removal. Hibernate normally issues the SQL `DELETE` when the persistence context is flushed.

Important: In the examples above, each Java class is a separate runnable demonstration. Run the insert example before running the search, update, or delete examples. Also, the factory is closed at the end of each program; do not reuse that closed factory within the same JVM.

Part 2: Advanced Hibernate Concepts

14. CRUD operation summary

Operation	Hibernate method	Purpose
Create	`persist()`	Makes a new entity persistent
Read	`find()`	Retrieves an entity by ID
Update managed entity	Modify its properties	Dirty checking detects changes
Update detached entity	`merge()`	Copies detached state into a managed instance
Delete	`remove()`	Marks a managed entity for deletion
Execute query	`createQuery()`	Creates an HQL query

What is `merge()`?

`merge()` is useful when working with an entity whose state is detached from the current persistence context.

```
Student detachedStudent = new Student(
        "Priya",
        "priya@gmail.com",
        "Python"
);

detachedStudent.setId(2L);

try (Session session =
        HibernateUtil.getSessionFactory().openSession()) {

    Transaction tx = session.beginTransaction();

    Student managedStudent = session.merge(detachedStudent);

    tx.commit();
}
```

This is only a simplified illustration. If the supplied ID already exists, Hibernate copies the state to the managed entity. If it does not exist, the provider may create a new persistent record, depending on entity state and identifier configuration.

Important: `merge()` returns the managed instance. The object passed to `merge()` does not itself become managed.

15. HQL queries with conditions

Suppose the database contains these students:

id	name	email	course
1	Rahul	rahul@gmail.com	Java
2	Priya	priya@gmail.com	Python
3	Aman	aman@gmail.com	Java
4	Neha	neha@gmail.com	Spring Boot

A. Find students by course

```
List<Student> students = session
        .createQuery(
            "FROM Student s WHERE s.course = :course",
            Student.class
        )
        .setParameter("course", "Java")
        .getResultList();
```

B. Find students by name

```
List<Student> students = session
        .createQuery(
            "FROM Student s WHERE s.name = :name",
            Student.class
        )
        .setParameter("name", "Rahul")
        .getResultList();
```

C. Find students whose name contains a keyword

```
List<Student> students = session
        .createQuery(
            "FROM Student s WHERE s.name LIKE :pattern",
            Student.class
        )
        .setParameter("pattern", "%a%")
        .getResultList();
```

D. Sort students by name

```
List<Student> students = session
        .createQuery(
            "FROM Student s ORDER BY s.name ASC",
            Student.class
        )
        .getResultList();
```

E. Count students

```
Long count = session
        .createQuery(
            "SELECT COUNT(s) FROM Student s",
            Long.class
        )
        .getSingleResult();

System.out.println("Total students: " + count);
```

Why use parameters?

Prefer:

```
.setParameter("course", "Java")
```

over building queries through string concatenation. Parameters reduce SQL injection risk and correctly bind values.

16. Hibernate pagination

Pagination retrieves records in smaller groups instead of loading an entire table.

```
List<Student> students = session
        .createQuery("FROM Student ORDER BY id", Student.class)
        .setFirstResult(0)
        .setMaxResults(5)
        .getResultList();
```

- `setFirstResult(0)` starts from the first result.
- `setMaxResults(5)` limits the result to five records.

To retrieve the next page of five records:

```
List<Student> students = session
        .createQuery("FROM Student ORDER BY id", Student.class)
        .setFirstResult(5)
        .setMaxResults(5)
        .getResultList();
```

This is useful when a table contains thousands or millions of records.

17. Entity lifecycle in Hibernate

Hibernate manages entities through different states.

Transient

New object, not associated with a session

`new Student()`

Persistent / Managed

Associated with an active persistence context

`persist()` or `find()`

Detached

No longer managed by the session

`detach()` or closing the session

Removed

Marked for deletion

`remove()`

Transient example

```
Student student = new Student(
        "Aman",
        "aman@gmail.com",
        "Java"
);
```

The object exists only in Java memory and is not yet managed by Hibernate.

Persistent example

```
session.persist(student);
```

The entity becomes managed by the session.

Detached example

```
session.detach(student);
```

Changes made after detaching the entity are not automatically tracked by that session.

Removed example

```
session.remove(student);
```

The entity is marked for deletion, subject to transaction commit.

18. Hibernate caching

Hibernate caching reduces unnecessary database queries.

First-level cache

The first-level cache is associated with the session's persistence context. It is enabled by default.

```
Student s1 = session.find(Student.class, 1L);
Student s2 = session.find(Student.class, 1L);
```

If the entity remains managed in the same session, the second lookup normally returns the same managed instance without another database query.

Second-level cache

The second-level cache is shared across sessions within a `SessionFactory`, provided it is configured with a supported cache provider.

It is not automatically enabled in a typical Hibernate setup.

Typical uses include relatively stable data that is read frequently.

Query cache

The query cache can cache query result information. It is separate from the second-level entity cache and requires explicit configuration.

Teaching tip: Learn session management and first-level caching before introducing second-level and query caching.

19. Lazy loading and eager loading

These concepts control when associated entities are fetched.

Lazy loading

Associated data is loaded when needed.

```
@ManyToOne(fetch = FetchType.LAZY)
private Department department;
```

Eager loading

Associated data is requested as part of loading the entity.

```
@ManyToOne(fetch = FetchType.EAGER)
private Department department;
```

The actual SQL strategy may use joins or additional queries.

Lazy loading	Eager loading
Defers loading associated data	Requests associated data immediately
Can reduce unnecessary retrieval	Can retrieve more data than required

Requires care when accessing data outside a session Can contribute to excessive queries or data loading

A common error is `LazyInitializationException`, which can occur when an uninitialized lazy association is accessed after the session is closed.

Solutions include fetching required associations within the transaction, using fetch joins, or retrieving DTO projections.

20. Hibernate relationships

Hibernate supports the same standard entity relationship mappings defined by Jakarta Persistence.

Relationship	Example
One-to-One	Person and Passport
One-to-Many	Department and Employees
Many-to-One	Many Employees belong to one Department
Many-to-Many	Students enroll in multiple Courses

Example: Department and Employee

Create `Department.java`:

```
package com.example;

import jakarta.persistence.*;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "departments")
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String departmentName;

    @OneToMany(mappedBy = "department")
    private List<Employee> employees = new ArrayList<>();

    public Department() {
    }

    public Department(String departmentName) {
        this.departmentName = departmentName;
    }

    public Long getId() {
        return id;
    }

    public String getDepartmentName() {
        return departmentName;
    }

    public void setDepartmentName(String departmentName) {
        this.departmentName = departmentName;
    }
```

```
    public List<Employee> getEmployees() {
        return employees;
    }
}
```

Create `Employee.java`:

```
package com.example;

import jakarta.persistence.*;

@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String employeeName;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id")
    private Department department;

    public Employee() {
    }

    public Employee(String employeeName, Department department) {
        this.employeeName = employeeName;
        this.department = department;
    }

    public Long getId() {
        return id;
    }

    public String getEmployeeName() {
        return employeeName;
    }

    public Department getDepartment() {
        return department;
    }
}
```

Add both mappings to `hibernate.cfg.xml`:

```
<mapping class="com.example.Department"/>
<mapping class="com.example.Employee"/>
```

Save a department and employees

```
try (Session session =
        HibernateUtil.getSessionFactory().openSession()) {

    Transaction tx = session.beginTransaction();

    Department department = new Department("IT");

    session.persist(department);

    Employee e1 = new Employee("Rahul", department);
    Employee e2 = new Employee("Priya", department);
```

```
    session.persist(e1);
    session.persist(e2);

    tx.commit();
}
```

The Department record is saved, followed by the two Employee records referencing it.

The `department_id` column in the `employees` table stores the foreign key.

For this example, no cascade is required because all three entities are explicitly persisted. In a larger application, consider the intended lifecycle of the related entities before adding cascade options.

21. Cascade types

Cascade propagates selected persistence operations from one entity to associated entities.

Cascade	Purpose
`PERSIST`	Propagates persistence
`MERGE`	Propagates merge
`REMOVE`	Propagates deletion
`REFRESH`	Propagates refresh
`DETACH`	Propagates detachment
`ALL`	Includes all listed cascade operations

Example:

```
@OneToOne(cascade = CascadeType.ALL)
private Passport passport;
```

This is appropriate only when the Passport's lifecycle should follow the owning entity's lifecycle. Be careful with cascading deletion on entities shared by multiple records.

22. Transactions in Hibernate

A transaction groups database operations into a unit of work.

For example, transferring money from one bank account to another should either complete both updates or roll them both back.

```
Transaction tx = null;

try (Session session =
        HibernateUtil.getSessionFactory().openSession()) {

    tx = session.beginTransaction();

    // Perform related database operations here.

    tx.commit();

} catch (Exception e) {

    if (tx != null && tx.isActive()) {
        tx.rollback();
    }
```

```
    throw e;
}
```

Why are transactions important?

- They maintain data consistency.
- They group related database operations.
- They enable rollback when an operation fails.
- They coordinate concurrent database operations.

In production code, handle exceptions according to the application's error-handling strategy rather than blindly rethrowing every exception.

23. Hibernate inheritance mapping

Hibernate supports mapping Java inheritance hierarchies to relational database structures.

The main strategies are:

Strategy	Description
`SINGLE_TABLE`	All hierarchy classes use one table
`JOINED`	Parent and subclass data use related tables
`TABLE_PER_CLASS`	Each concrete class uses its own table

Example: SINGLE_TABLE

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "employee_type")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
}
```

Subclass:

```
@Entity
@DiscriminatorValue("MANAGER")
public class Manager extends Employee {

    private String departmentManaged;
}
```

The Employee and Manager data are stored in one table, with a discriminator column identifying the subclass.

For `SINGLE_TABLE`, subclass-specific columns generally need to allow `NULL` for rows representing other subclasses.

Part 3: Hibernate and JPA APIs

24. Hibernate Session vs JPA EntityManager

Hibernate implements Jakarta Persistence and provides its own native API.

Hibernate API	JPA API
`SessionFactory`	`EntityManagerFactory`
`Session`	`EntityManager`
`session.persist()`	`entityManager.persist()`
`session.find()`	`entityManager.find()`
`session.remove()`	`entityManager.remove()`
`session.createQuery()`	`entityManager.createQuery()`

Many operations are similar, but Hibernate's `Session` also exposes provider-specific functionality.

Which should you learn first?

For students, a useful learning path is:

1. Understand direct Hibernate CRUD using `Session`.
2. Learn the JPA specification and `EntityManager`.
3. Learn Spring Data JPA repositories.
4. Explore Hibernate-specific performance and caching features.

This sequence helps distinguish standard persistence concepts from Hibernate-specific APIs.

25. Hibernate with Spring Boot

In a Spring Boot project, you usually do not configure `SessionFactory` manually. Spring Boot configures Hibernate through Spring Data JPA.

Maven dependencies:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>
```

Example configuration:

```
spring.datasource.url=jdbc:mysql://localhost:3306/hibernate_db
spring.datasource.username=root
spring.datasource.password=YOUR_MYSQL_PASSWORD

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Repository:

```
public interface StudentRepository
        extends JpaRepository<Student, Long> {
```

```
}
```

Spring Data JPA supplies the implementation of the repository methods, while Hibernate performs the ORM work underneath.

Remember: `JpaRepository` belongs to Spring Data JPA, not directly to Hibernate.

Part 4: Common Hibernate Errors

26. Troubleshooting

Error	Likely cause	What to check
`Communications link failure`	MySQL is unavailable or unreachable	Start MySQL and verify port and URL
`Access denied for user`	Incorrect database credentials or permissions	Check username and password
`Unknown database`	Database was not created	Run `CREATE DATABASE hibernate_db`
`No Persistence provider`	Missing or incorrect provider configuration	Check dependencies and persistence configuration
`Not a managed type`	Entity not registered or missing `@Entity`	Check entity mapping
`No identifier specified`	Entity lacks a primary key	Add `@Id`
`LazyInitializationException`	Lazy relationship accessed outside the session	Load data within the appropriate transaction
`ConstraintViolationException`	A database constraint was violated	Check duplicate values and required columns
`detached entity passed to persist`	Detached entity passed to an operation expecting a new entity	Review entity state and use `merge()` when appropriate
`package jakarta.persistence does not exist`	Jakarta Persistence classes are unavailable on the classpath	Check Hibernate dependencies and Maven reload

`javax.persistence` VS `jakarta.persistence`

Hibernate 6 uses Jakarta Persistence.

Use:

```
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
```

Do not use the old `javax.persistence` imports in a Hibernate 6 application.

Part 5: Student Practice

27. Practical exercises

Hibernate lab checklist

0/15 complete

- [ ] 1. Create a Product entity with id, name, price and quantity.
- [ ] 2. Insert five products into MySQL using Hibernate.
- [ ] 3. Retrieve a product using its primary key.
- [ ] 4. Display all products using HQL.
- [ ] 5. Update a product price.
- [ ] 6. Delete a product by ID.
- [ ] 7. Find products whose price is greater than 1000.
- [ ] 8. Sort products by name using HQL.
- [ ] 9. Count the total number of products.
- [ ] 10. Implement pagination using setFirstResult() and setMaxResults().
- [ ] 11. Create Department and Employee entities with a Many-to-One relationship.
- [ ] 12. Demonstrate the transient, persistent and detached states.
- [ ] 13. Demonstrate dirty checking by updating a managed entity.
- [ ] 14. Use a transaction to perform two database operations.
- [ ] 15. Investigate the N+1 query problem and a suitable fetch strategy.

Reset checklist

28. Interview questions

1. What is Hibernate?
2. What is ORM?
3. What is the difference between Hibernate and JDBC?
4. What is the difference between JPA and Hibernate?
5. What is a `SessionFactory`?
6. What is a Hibernate `Session`?
7. What is the purpose of `@Entity`?
8. What is the difference between `persist()` and `merge()`?
9. What is dirty checking?
10. What is the first-level cache?
11. What is the second-level cache?
12. What is HQL?
13. What is the difference between HQL and SQL?
14. What is lazy loading?
15. What is eager loading?
16. What is the N+1 query problem?
17. What are cascade types?
18. What is the purpose of `@JoinColumn`?
19. How does Hibernate manage transactions?
20. What are the inheritance mapping strategies?

29. Suggested teaching schedule

Session	Topics	Duration
1	ORM introduction, Hibernate architecture, Maven setup	60 min
2	Configuration, entities, annotations and table creation	60 min
3	Insert, select, update and delete	90 min
4	HQL, conditions, sorting and pagination	60 min
5	Entity lifecycle, caching and fetching	60 min

30. Final summary

The most important concepts in Hibernate are:

- ORM: Maps Java objects to database tables.
- Entity: Represents persistent data.
- SessionFactory: Creates Hibernate sessions.
- Session: Manages entity operations and persistence context.
- Transaction: Groups database operations.
- HQL: Queries entities using object-oriented syntax.
- Dirty checking: Automatically detects changes to managed entities.
- Caching: Reduces repeated database access.
- Relationships: Connect related entity objects.
- Lazy loading: Defers loading associated data.
- JPA integration: Provides a standard persistence API implemented by Hibernate.

Official learning resources

- Hibernate ORM Documentation
- Jakarta Persistence Specification
- Spring Data JPA Documentation

Recommended practice: First execute the standalone Hibernate CRUD project using MySQL. Then add HQL queries and entity relationships. Finally, implement the same functionality using Spring Data JPA to understand how Spring simplifies Hibernate-based persistence.

Convert to PDF
Basic RequestMapping with Example Response Entity

Create a complete detailed tutorial with example

Complete Tutorial: Basic `@RequestMapping` and `ResponseEntity` in Spring Boot

For beginners | Java | Spring Boot | REST API | HTTP Methods | JSON Responses

This tutorial explains `@RequestMapping` and `ResponseEntity` from the basics, with definitions, syntax, examples, complete runnable code, expected outputs, and practice exercises.

1. What is `@RequestMapping`?

Definition: `@RequestMapping` is a Spring MVC annotation used to map HTTP requests to controller classes or methods.

When a client sends a request from a browser, Postman, or a frontend application, Spring uses request mappings to determine which controller method should handle it.

For example, when a client requests:

```
GET http://localhost:8080/hello
```

Spring can execute a controller method mapped to `/hello`.

Basic example

```
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    @RequestMapping("/hello")
    public String hello() {
        return "Hello, Spring Boot!";
    }
}
```

Expected response:

```
Hello, Spring Boot!
```

Here:

- `@RestController` marks the class as a REST controller.
- `@RequestMapping("/hello")` maps requests to the `/hello` path.
- `hello()` returns the response body.

By default, this mapping accepts all standard HTTP methods unless you restrict it. In REST APIs, it is usually better to specify the intended HTTP method explicitly.

2. What are the important annotations?

Annotation	Purpose
`@RestController`	Creates a controller whose return values are written to the response body
`@Controller`	Defines a Spring MVC controller, often used for HTML views
`@RequestMapping`	Maps HTTP requests to controller classes or methods
`@GetMapping`	Maps HTTP GET requests
`@PostMapping`	Maps HTTP POST requests
`@PutMapping`	Maps HTTP PUT requests
`@PatchMapping`	Maps HTTP PATCH requests
`@DeleteMapping`	Maps HTTP DELETE requests
`@PathVariable`	Reads a value from the URL path
`@RequestParam`	Reads a query parameter
`@RequestBody`	Reads request-body data, commonly JSON
`@ResponseBody`	Writes a method's return value to the response body
`@ResponseStatus`	Specifies an HTTP status for a response
`ResponseEntity`	Controls the HTTP status, headers, and response body

3. Understanding `@RequestMapping` syntax

The general syntax is:

```
@RequestMapping(
    value = "/path",
    method = RequestMethod.GET
)
```

The main attributes are:

Attribute	Meaning	Example
`value`	URL path or paths	`value = "/students"`
`path`	Alternative to `value`	`path = "/students"`
`method`	Allowed HTTP method	`method = RequestMethod.GET`
`consumes`	Supported request content type	`consumes = "application/json"`
`produces`	Response content type	`produces = "application/json"`
`params`	Matches query parameter conditions	`params = "course"`
`headers`	Matches HTTP header conditions	`headers = "X-API-Version=1"`

For example:

```
@RequestMapping(
    value = "/students",
    method = RequestMethod.GET,
    produces = "application/json"
)
public String getStudents() {
    return "Student list";
}
```

The method is mapped to GET requests for `/students`, and the response is declared as JSON. Since the returned value is a plain string, it is not a JSON array or JSON object.

4. Class-level and method-level `@RequestMapping`

Spring allows `@RequestMapping` at both the class and method levels.

Class-level mapping

```
@RequestMapping("/students")
```

Method-level mapping

```
@GetMapping("/{id}")
```

Combined endpoint

`GET /students/10`

Example:

```
@RestController
@RequestMapping("/students")
public class StudentController {

    @GetMapping("/{id}")
    public String getStudent() {
        return "Student details";
    }
}
```

The combined path is `/students/{id}`. The `{id}` placeholder allows different student IDs.

To read the ID, add `@PathVariable`:

```
@GetMapping("/{id}")
public String getStudent(@PathVariable Long id) {
    return "Student ID: " + id;
}
```

Request:

```
GET http://localhost:8080/students/10
```

Response:

```
Student ID: 10
```

5. Different HTTP methods in Spring Boot

REST APIs use HTTP methods to describe the operation a client wants to perform.

HTTP method	Common purpose	Spring annotation
GET	Retrieve data	`@GetMapping`
POST	Create a resource	`@PostMapping`
PUT	Replace or update a resource	`@PutMapping`
PATCH	Partially update a resource	`@PatchMapping`
DELETE	Delete a resource	`@DeleteMapping`

These annotations are shortcuts for `@RequestMapping` with the corresponding HTTP method.

For example, these mappings are equivalent:

```
@RequestMapping(
    value = "/hello",
    method = RequestMethod.GET
)
@GetMapping("/hello")
```

Recommendation: Use `@GetMapping`, `@PostMapping`, `@PutMapping`, and `@DeleteMapping` for ordinary REST endpoints because they make the intended HTTP method clear.

Part 2: What is `ResponseEntity`?

6. Definition of `ResponseEntity`

`ResponseEntity` is a Spring class that represents an HTTP response, including its body, status code, and headers.

It is useful when a controller needs to return different HTTP status codes depending on the outcome of an operation.

For example:

- `200 OK` — Request succeeded.
- `201 Created` — A resource was created.
- `204 No Content` — Request succeeded without a response body.
- `400 Bad Request` — Request data is invalid.
- `404 Not Found` — Requested resource does not exist.
- `500 Internal Server Error` — An unexpected server error occurred.

Without `ResponseEntity`

```
@GetMapping("/hello")
public String hello() {
    return "Hello Student";
}
```

This returns the body as a string. Spring normally sends `200 OK` when the request succeeds.

With `ResponseEntity`

```
@GetMapping("/hello")
public ResponseEntity<String> hello() {
    return ResponseEntity
            .ok("Hello Student");
}
```

This explicitly creates an HTTP response with status `200 OK` and the specified body.

Basic syntax

```
ResponseEntity<BodyType>
```

Examples:

```
ResponseEntity<String>
ResponseEntity<Student>
ResponseEntity<List<Student>>
ResponseEntity<Void>
```

The generic type describes the body type, not the HTTP status code.

7. Important `ResponseEntity` methods

Method	Purpose
`ResponseEntity.ok(body)`	Returns `200 OK` with a body
`ResponseEntity.ok().build()`	Returns `200 OK` without a body
`ResponseEntity.status(code).body(body)`	Returns a specified status and body
`ResponseEntity.created(uri).body(body)`	Returns `201 Created` with a Location header
`ResponseEntity.noContent().build()`	Returns `204 No Content`
`ResponseEntity.badRequest().body(body)`	Returns `400 Bad Request`
`ResponseEntity.notFound().build()`	Returns `404 Not Found`
`ResponseEntity.internalServerError().body(body)`	Returns `500 Internal Server Error`

Example: Different responses

```
@GetMapping("/success")
```

```
public ResponseEntity<String> success() {
    return ResponseEntity.ok("Operation successful");
}
```

Response:

```
200 OK
Operation successful
```

Example with a not-found response:

```
@GetMapping("/missing")
public ResponseEntity<String> missing() {
    return ResponseEntity
            .status(404)
            .body("Record not found");
}
```

Response:

```
404 Not Found
Record not found
```

For a real API, the response body for errors is often represented by a structured JSON object rather than a plain string.

8. How `@RequestMapping` and `ResponseEntity` work together

These two features serve different purposes.

`@RequestMapping`	`ResponseEntity`
Maps incoming requests	Represents outgoing responses
Determines which method handles the request	Determines response status, headers, and body
Works with paths and HTTP methods	Works with response data
Example: `@GetMapping("/students")`	Example: `ResponseEntity.ok(students)`

Example:

```
@GetMapping("/students")
public ResponseEntity<List<Student>> getStudents() {
    List<Student> students = List.of(
        new Student(1L, "Rahul"),
        new Student(2L, "Priya")
    );

    return ResponseEntity.ok(students);
}
```

The mapping handles the GET request. `ResponseEntity` returns the list with `200 OK`.

Part 3: Complete Runnable Example

9. Create a Spring Boot project

Use Spring Initializr and choose:

Setting	Value
Project	Maven
Language	Java
Spring Boot	A compatible Spring Boot 3.x release
Group	`com.example`
Artifact	`requestmapping-demo`
Java	17
Dependency	Spring Web

No database is required for this example. We will store students in an in-memory list.

Project structure

```
requestmapping-demo/
│
├── pom.xml
│
└── src/
    └── main/
        └── java/
            └── com/example/requestmappingdemo/
                ├── RequestmappingDemoApplication.java
                ├── Student.java
                └── StudentController.java
```

10. Student.java — Create the model class

```
package com.example.requestmappingdemo;

public class Student {

    private Long id;
    private String name;
    private String course;

    public Student() {
    }

    public Student(Long id, String name, String course) {
        this.id = id;
        this.name = name;
        this.course = course;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getCourse() {
```

```
        return course;
    }

    public void setCourse(String course) {
        this.course = course;
    }
}
```

Explanation

- `Student` is a Java model class.
- `id` identifies the student.
- `name` stores the student's name.
- `course` stores the course name.
- The no-argument constructor and getters/setters support object creation and JSON conversion.

This is an ordinary Java object. Because this example has no database, it does not need JPA's `@Entity` annotation.

11. StudentController.java — Create REST endpoints

The following example includes GET, POST, PUT, DELETE, and several response status codes.

```
package com.example.requestmappingdemo;

import java.net.URI;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/students")
public class StudentController {

    private final List<Student> students = new ArrayList<>(
        List.of(
            new Student(1L, "Rahul", "Java"),
            new Student(2L, "Priya", "Python"),
            new Student(3L, "Aman", "Spring Boot")
        )
    );

    private long nextId = 4L;

    // GET: Retrieve all students
    @GetMapping
    public ResponseEntity<List<Student>> getAllStudents() {
        return ResponseEntity.ok(students);
    }

    // GET: Retrieve one student
    @GetMapping("/{id}")
    public ResponseEntity<Student> getStudentById(
            @PathVariable Long id) {

        Optional<Student> student = students.stream()
                .filter(s -> s.getId().equals(id))
                .findFirst();
```

```
        return student
                .map(ResponseEntity::ok)
                .orElseGet(() -> ResponseEntity.notFound().build());
    }

    // POST: Create a student
    @PostMapping
    public ResponseEntity<Student> addStudent(
            @RequestBody Student student) {

        student.setId(nextId++);
        students.add(student);

        URI location = URI.create("/students/" + student.getId());

        return ResponseEntity.created(location).body(student);
    }

    // PUT: Update an existing student
    @PutMapping("/{id}")
    public ResponseEntity<Student> updateStudent(
            @PathVariable Long id,
            @RequestBody Student updatedStudent) {

        Optional<Student> existing = students.stream()
                .filter(s -> s.getId().equals(id))
                .findFirst();

        if (existing.isEmpty()) {
            return ResponseEntity.notFound().build();
        }

        Student student = existing.get();
        student.setName(updatedStudent.getName());
        student.setCourse(updatedStudent.getCourse());

        return ResponseEntity.ok(student);
    }

    // DELETE: Remove a student
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteStudent(
            @PathVariable Long id) {

        boolean removed = students.removeIf(
                s -> s.getId().equals(id)
        );

        if (!removed) {
            return ResponseEntity.notFound().build();
        }

        return ResponseEntity.noContent().build();
    }

    // GET: Example of a custom response
    @GetMapping("/message")
    public ResponseEntity<String> getMessage() {
        return ResponseEntity.ok("Student API is working");
    }
}
```

Explanation of the controller

```
@RestController
```

Spring registers the class as a REST controller. Returned objects are written to the HTTP response body, normally as JSON when the appropriate converter is available.

`@RequestMapping("/students")`

Defines the common path for all methods in the class.

`@GetMapping`

Retrieves all students.

`@GetMapping("/{id}")`

Retrieves one student based on the path variable.

`@PathVariable Long id`

Extracts the ID from the URL, such as `/students/2`.

`@PostMapping` and `@RequestBody`

Accepts JSON data and converts it to a Student object.

`@PutMapping("/{id}")`

Updates the matching student.

`@DeleteMapping("/{id}")`

Removes a student and returns `204 No Content` when successful.

`ResponseEntity.created(location).body(student)`

Returns `201 Created`, includes a `Location` header identifying the new resource, and returns the created student.

`ResponseEntity.notFound().build()`

Returns `404 Not Found` with no response body.

`ResponseEntity.noContent().build()`

Returns `204 No Content` with no response body.

This is a teaching example using an in-memory list. Data disappears when the application restarts, and the list is not designed for concurrent production use.

12. Main application class

```
package com.example.requestmappingdemo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class RequestmappingDemoApplication {

    public static void main(String[] args) {
```

```
        SpringApplication.run(
            RequestmappingDemoApplication.class,
            args
        );
    }
}
```

13. Run the application

In the project directory, execute:

```
mvn spring-boot:run
```

When the application starts successfully, the server listens on port `8080` by default.

You can test the GET endpoints directly in your browser. Use Postman or another HTTP client to test POST, PUT, and DELETE.

Part 4: Test Every Endpoint

14. GET — Retrieve all students

Request:

```
GET http://localhost:8080/students
```

Expected status:

```
200 OK
```

Response body:

```
[
  {
    "id": 1,
    "name": "Rahul",
    "course": "Java"
  },
  {
    "id": 2,
    "name": "Priya",
    "course": "Python"
  },
  {
    "id": 3,
    "name": "Aman",
    "course": "Spring Boot"
  }
]
```

15. GET — Retrieve a student by ID

Request:

```
GET http://localhost:8080/students/2
```

Expected status:

```
200 OK
```

Response:

```
{
  "id": 2,
  "name": "Priya",
  "course": "Python"
}
```

If you request `/students/99`, the response is `404 Not Found`.

16. POST — Add a new student

In Postman:

- Method: `POST`
- URL: `http://localhost:8080/students`
- Select Body → raw → JSON

Request body:

```
{
  "name": "Neha",
  "course": "Spring Boot"
}
```

Expected status:

```
201 Created
```

Response body:

```
{
  "id": 4,
  "name": "Neha",
  "course": "Spring Boot"
}
```

The response includes a header similar to:

```
Location: /students/4
```

This identifies the newly created resource.

17. PUT — Update a student

Request:

```
PUT http://localhost:8080/students/2
```

Request body:

```
{
  "name": "Priya Sharma",
  "course": "Advanced Java"
}
```

Expected status:

```
200 OK
```

Response:

```
{
  "id": 2,
  "name": "Priya Sharma",
  "course": "Advanced Java"
}
```

If the ID does not exist, the controller returns `404 Not Found`.

18. DELETE — Delete a student

Request:

```
DELETE http://localhost:8080/students/2
```

Expected status:

```
204 No Content
```

There is no response body. If the student does not exist, the controller returns `404 Not Found`.

19. GET — Return a custom message

Request:

```
GET http://localhost:8080/students/message
```

Expected response:

```
200 OK
Student API is working
```

In this case, the response body is plain text because the method returns a string and does not explicitly declare a JSON response type.

Part 5: Understanding HTTP Status Codes

20. Common status codes with `ResponseEntity`

Status	Meaning	Typical use
`200 OK`	Request succeeded	GET or successful update
`201 Created`	Resource created	Successful POST
`204 No Content`	Request succeeded without a body	Successful DELETE
`400 Bad Request`	Invalid request	Failed input validation
`401 Unauthorized`	Authentication is required or invalid	Login/token failure
`403 Forbidden`	Request is not permitted	Insufficient permissions
`404 Not Found`	Resource not found	Unknown student ID

`409 Conflict`	Request conflicts with current state	Duplicate or conflicting data
`500 Internal Server Error`	Unexpected server failure	Unhandled server error

Example: Return a bad request response

```
@GetMapping("/error-example")
public ResponseEntity<String> errorExample() {
    return ResponseEntity
            .badRequest()
            .body("Invalid request");
}
```

This returns status `400 Bad Request`.

Example: Return a custom status

```
@GetMapping("/custom-status")
public ResponseEntity<String> customStatus() {
    return ResponseEntity
            .status(202)
            .body("Request accepted for processing");
}
```

This returns `202 Accepted`. Use this status when processing has been accepted but is not yet complete.

Part 6: Additional Request Mapping Features

21. Using `@RequestParam`

`@RequestParam` reads query parameters.

Example URL:

```
http://localhost:8080/students/search?course=Java
```

Controller method:

```
@GetMapping("/search")
public ResponseEntity<String> searchByCourse(
        @RequestParam String course) {

    return ResponseEntity.ok(
        "Searching students in course: " + course
    );
}
```

Response:

```
Searching students in course: Java
```

You can also provide a default value:

```
@GetMapping("/welcome")
public ResponseEntity<String> welcome(
        @RequestParam(defaultValue = "Student") String name) {

    return ResponseEntity.ok("Welcome " + name);
}
```

Request:

```
GET /students/welcome
```

Response:

```
Welcome Student
```

22. Using `@PathVariable` VS `@RequestParam`

Feature	`@PathVariable`	`@RequestParam`
Source	URL path	Query string or form parameter
Example	`/students/10`	`/students?id=10`
Common purpose	Identify a specific resource	Filtering or optional parameters
Java example	`@PathVariable Long id`	`@RequestParam Long id`

Both can be used to receive values from the request, but they express different URL designs.

23. Using `produces` and `consumes`

These attributes describe the content types accepted or returned by a mapping.

Example: JSON response

```
@GetMapping(
    value = "/json",
    produces = "application/json"
)
public ResponseEntity<Student> getJson() {
    return ResponseEntity.ok(
        new Student(10L, "Rahul", "Java")
    );
}
```

Example: JSON request

```
@PostMapping(
    value = "/json",
    consumes = "application/json"
)
public ResponseEntity<Student> createJson(
        @RequestBody Student student) {

    student.setId(100L);

    return ResponseEntity
            .status(201)
            .body(student);
}
```

`consumes` helps Spring match the request's `Content-Type`, while `produces` helps match the client's accepted response media type and describes the response representation.

24. Mapping multiple paths

A controller method can support multiple paths.

```
@RequestMapping(
    value = {"/hello", "/welcome"},
    method = RequestMethod.GET
)
public ResponseEntity<String> greet() {
    return ResponseEntity.ok("Welcome to Spring Boot");
}
```

Both endpoints return the same response:

- `/hello`
- `/welcome`

For GET-only endpoints, `@GetMapping({"/hello", "/welcome"})` is a more concise alternative.

Part 7: Common Errors and Solutions

25. Troubleshooting

Problem	Possible cause	Solution
`404 Not Found`	Incorrect URL or path mapping	Check the controller path and request URL
`405 Method Not Allowed`	Wrong HTTP method	Use the method supported by the endpoint
`400 Bad Request`	Invalid JSON or incorrect data type	Check request body and field types
`415 Unsupported Media Type`	Incorrect request `Content-Type`	Set `Content-Type: application/json`
`406 Not Acceptable`	Response type does not match `Accept`	Check the requested and produced media types
`500 Internal Server Error`	Unhandled exception	Inspect the application logs
`No qualifying bean`	Controller or dependency not registered	Check annotations and component scanning
JSON not returned	Incorrect controller annotation or response configuration	Use `@RestController` or `@ResponseBody`

26. Practice exercises

Try implementing these tasks in the project:

Student task checklist

0 of 10

- [ ] 1. Create a GET endpoint that returns your name.
- [ ] 2. Create a GET endpoint that returns a list of three subjects.
- [ ] 3. Create a GET endpoint using @PathVariable to display an employee ID.
- [ ] 4. Create a GET endpoint using @RequestParam to accept a city.
- [ ] 5. Create a POST endpoint that accepts a Student JSON object.
- [ ] 6. Return 201 Created after inserting a student.
- [ ] 7. Return 404 Not Found when a student ID is missing.

- [ ] 8. Create a PUT endpoint to update student details.
- [ ] 9. Create a DELETE endpoint that returns 204 No Content.
- [ ] 10. Create a custom response using ResponseEntity with a Location header.

Reset checklist

27. Interview questions

1. What is `@RequestMapping`?
2. What is the difference between `@RequestMapping` and `@GetMapping`?
3. What is the purpose of `@RestController`?
4. What is `ResponseEntity`?
5. What are the advantages of using `ResponseEntity`?
6. What is the difference between `@PathVariable` and `@RequestParam`?
7. What is the purpose of `@RequestBody`?
8. What is the difference between `200 OK` and `201 Created`?
9. Why does a DELETE endpoint often return `204 No Content`?
10. What is the purpose of `consumes` and `produces`?
11. What is the difference between `@Controller` and `@RestController`?
12. How do you return `404 Not Found` using `ResponseEntity`?

28. Final summary

- `@RequestMapping` maps HTTP requests to controller classes and methods.
- `@GetMapping`, `@PostMapping`, `@PutMapping`, and `@DeleteMapping` are specialized request-mapping annotations.
- `ResponseEntity` controls the HTTP response status, headers, and body.
- `@PathVariable` reads URL path values.
- `@RequestParam` reads query parameters.
- `@RequestBody` converts request-body data into Java objects.
- HTTP status codes communicate the result of the request to the client.

Recommended classroom sequence: Start with a simple GET endpoint, then introduce class-level and method-level mappings, followed by `ResponseEntity`, JSON request bodies, CRUD operations, and HTTP status codes.

Official references:

- Spring MVC — Request Mapping
- Spring `ResponseEntity` API
- Building a RESTful Web Service

Convert to PDF
RequestMapping and GetMapping is same

Yes, `@RequestMapping` and `@GetMapping` can do the same thing when mapping GET requests, but they are not exactly the same.

1. Example using `@RequestMapping`

```
@RestController
public class StudentController {

    @RequestMapping(
```

```
        value = "/students",
        method = RequestMethod.GET
    )
    public String getStudents() {
        return "List of Students";
    }
}
```

2. Example using `@GetMapping`

```
@RestController
public class StudentController {

    @GetMapping("/students")
    public String getStudents() {
        return "List of Students";
    }
}
```

Both examples handle:

```
GET http://localhost:8080/students
```

Response:

```
List of Students
```

3. Difference between them

Feature	`@RequestMapping`	`@GetMapping`
Purpose	Maps requests for one or more HTTP methods	Specifically maps GET requests
Default HTTP method	Not restricted to a specific method	GET only
Syntax	More configurable	Shorter and simpler
Supports POST, PUT, DELETE	Yes	No
Recommended use	General or class-level mappings	GET endpoints

4. One important difference

Consider this code:

```
@RequestMapping("/students")
public String getStudents() {
    return "Students";
}
```

Because no HTTP method is specified, the mapping can match multiple HTTP methods.

But:

```
@GetMapping("/students")
public String getStudents() {
    return "Students";
}
```

This method handles only GET requests.

5. Other specialized mapping annotations

Annotation	Equivalent mapping
`@GetMapping("/students")`	`@RequestMapping(value="/students", method=RequestMethod.GET)`
`@PostMapping("/students")`	`@RequestMapping(value="/students", method=RequestMethod.POST)`
`@PutMapping("/students")`	`@RequestMapping(value="/students", method=RequestMethod.PUT)`
`@DeleteMapping("/students")`	`@RequestMapping(value="/students", method=RequestMethod.DELETE)`
`@PatchMapping("/students")`	`@RequestMapping(value="/students", method=RequestMethod.PATCH)`

Conclusion: Both can map GET requests to the same controller method. For beginners, I recommend using `@GetMapping` for GET operations and the corresponding specialized annotations for other HTTP methods. Use `@RequestMapping` when you need more general request-mapping configuration or a common base path at the controller class level.

Spring Data JPA: Complete CRUD Tutorial

This tutorial covers:

1. What is JPA?
2. What is Spring Data JPA?
3. Entity
4. Repository
5. CRUD Operations (Create, Read, Update, Delete)
6. Complete Example

---

1. What is JPA?

JPA (Java Persistence API) is a specification that defines how Java objects are stored and retrieved from a database.

Example:

Java

Employee emp = new Employee();

JPA helps save this Java object into a database table.

---

2. What is Spring Data JPA?

Spring Data JPA is built on top of:

Plain Text

Spring Boot

↓

Spring Data JPA

↓

Hibernate

↓

Database

Benefits:

- Less code
- No SQL for basic CRUD
- Ready-made methods
- Automatic implementation

Project Structure

Plain Text

src/main/java

│

├── entity

│ Employee.java

│

├── repository

│ EmployeeRepository.java

│

├── controller

│ EmployeeController.java

│

└── EmployeeApplication.java

Step 1: Dependencies

pom.xml

XML

```
<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
<groupId>com.mysql</groupId>
<artifactId>mysql-connector-j</artifactId>
</dependency>

</dependencies>
```

Step 2: Configure Database

application.properties

.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/employeedb
spring.datasource.username=root
spring.datasource.password=root

spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.show-sql=true
```

Step 3: Create Entity

Employee.java

Java

```
package com.example.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "employees")
public class Employee {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

private String name;

private String department;

private double salary;

// Constructors
public Employee() {
}

public Employee(Long id, String name,
String department,
double salary) {
this.id = id;
this.name = name;
```

```
this.department = department;
this.salary = salary;
}

// Getters and Setters

public Long getId() {
return id;
}

public void setId(Long id) {
this.id = id;
}

public String getName() {
return name;
}

public void setName(String name) {
this.name = name;
}

public String getDepartment() {
return department;
}

public void setDepartment(String department) {
this.department = department;
}

public double getSalary() {
return salary;
}
```

```
public void setSalary(double salary) {
this.salary = salary;
}
}
```

Understanding Entity Annotations

Java

@Entity

Marks class as database table.

Java

@Table(name="employees")

Creates table:

SQL

employees

Java

@Id

Primary Key.

Java

@GeneratedValue(strategy=GenerationType.IDENTITY)

Auto increments ID.

Plain Text

1

2

3

4

...

Step 4: Create Repository

EmployeeRepository.java

Java

```
package com.example.repository;


import com.example.entity.Employee;
import org.springframework.data.jpa.repository.JpaRepository;


public interface EmployeeRepository
extends JpaRepository<Employee,Long> {


}
```

What is Repository?

Repository acts as a bridge between:

Plain Text

```
Java Application
↓
Repository
↓
Database
```

By extending:

Java

```
JpaRepository<Employee,Long>
```

you automatically get:

Java

```
save()
findAll()
```

findById()

deleteById()

count()

existsById()

No implementation needed.

Step 5: Create Controller

EmployeeController.java

Java

```
package com.example.controller;

import com.example.entity.Employee;
import com.example.repository.EmployeeRepository;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/employees")
public class EmployeeController {

private final EmployeeRepository repository;

public EmployeeController(EmployeeRepository repository) {
this.repository = repository;
}

// CREATE
@PostMapping
public Employee addEmployee(
@RequestBody Employee employee) {
```

```
return repository.save(employee);
}

// READ ALL
@GetMapping
public List<Employee> getAllEmployees() {
return repository.findAll();
}

// READ BY ID
@GetMapping("/{id}")
public Employee getEmployee(
@PathVariable Long id) {
return repository.findById(id)
.orElse(null);
}

// UPDATE
@PutMapping("/{id}")
public Employee updateEmployee(
@PathVariable Long id,
@RequestBody Employee employee) {

Employee existing =
repository.findById(id)
.orElse(null);

if (existing != null) {

existing.setName(employee.getName());
existing.setDepartment(
employee.getDepartment());
```

```
existing.setSalary(
employee.getSalary());

return repository.save(existing);
}

return null;
}

// DELETE
@DeleteMapping("/{id}")
public String deleteEmployee(
@PathVariable Long id) {

repository.deleteById(id);

return "Employee Deleted";
}
}
```

CRUD Operations

1. Create Employee

Request

HTTP

POST /employees

Body:

JSON

```
{
"name":"Mohammad",
"department":"IT",
"salary":50000
```

}

Internally

Java

repository.save(employee);

SQL Generated:

SQL

INSERT INTO employees

(name,department,salary)

VALUES

('Mohammad','IT',50000);

---

2. Get All Employees

Request

HTTP

GET /employees

Code

Java

repository.findAll();

SQL:

SQL

SELECT * FROM employees;

---

3. Get Employee By ID

Request

HTTP

GET /employees/1

Code

Java

repository.findById(1L);

SQL:

SQL

SELECT *

FROM employees

WHERE id=1;

---

4. Update Employee

Request

HTTP

PUT /employees/1

Body:

JSON

{

"name":"Mohammad Khan",

"department":"HR",

"salary":60000

}

Code

Java

repository.save(existingEmployee);

SQL:

SQL

UPDATE employees

SET name='Mohammad Khan'

WHERE id=1;

---

5. Delete Employee

Request

HTTP

DELETE /employees/1

Code

Java

repository.deleteById(1L);

SQL:

SQL

DELETE FROM employees

WHERE id=1;

Common JpaRepository Methods

Java

save(entity)

Insert or Update

Java

findAll()

Get all records

Java

findById(id)

Get one record

Java

deleteById(id)

Delete record

Java

count()

Count records

Java

existsById(id)

Check whether record exists

Interview Questions

What is JPA?

JPA is a Java specification for Object Relational Mapping (ORM) to persist Java objects into a database.

What is Hibernate?

Hibernate is the most popular implementation of JPA.

What is Spring Data JPA?

Spring Data JPA provides repositories and reduces boilerplate code for database operations.

What is an Entity?

An Entity is a Java class mapped to a database table using `@Entity`.

What is JpaRepository?

`JpaRepository` is an interface that provides ready-made CRUD methods without writing SQL.

What does `save()` do?

- Insert if ID does not exist.
- Update if ID already exists.

Quick Flow Diagram

Plain Text

Client/Postman

|

v

EmployeeController

|

v

EmployeeRepository

|

v

Spring Data JPA

|

v

Hibernate

|

v

MySQL Database

This is the basic Spring Boot + Spring Data JPA CRUD application that is commonly asked in interviews and used as the foundation for larger projects.

Provide your feedback on BizChat

You said:
I have to check this code via postmal

Copilot said:

Yes. After running your Spring Boot application, you can test all CRUD APIs in **Postman**.

Assume your application runs on:

Plain Text

http://localhost:8080

and your controller has:

Java

@RequestMapping("/employees")

---

1. CREATE Employee

Method

HTTP

POST

URL

Plain Text

http://localhost:8080/employees

Body → raw → JSON

JSON

```
{
"name":"Mohammad",
"department":"IT",
"salary":50000
}
```

Expected Response

JSON

```
{
"id":1,
"name":"Mohammad",
"department":"IT",
"salary":50000.0
}
```

---

2. GET All Employees

Method

HTTP

GET

URL

Plain Text

http://localhost:8080/employees

Response

JSON

[
{
"id":1,
"name":"Mohammad",
"department":"IT",
"salary":50000.0
}
]

---

3. GET Employee By ID

Method

HTTP

GET

URL

Plain Text

http://localhost:8080/employees/1

Response

JSON

{
"id":1,
"name":"Mohammad",
"department":"IT",
"salary":50000.0
}

---

4. UPDATE Employee

Method

HTTP

PUT

URL

Plain Text

http://localhost:8080/employees/1

Body

JSON

{

"name":"Mohammad Khan",

"department":"HR",

"salary":60000

}

Response

JSON

{

"id":1,

"name":"Mohammad Khan",

"department":"HR",

"salary":60000.0

}

---

5. DELETE Employee

Method

HTTP

DELETE

URL

Plain Text

http://localhost:8080/employees/1

Response

Plain Text

Employee Deleted

If Spring Security is Enabled

If you have configured Spring Security, go to:

Authorization → Basic Auth

Username:

Plain Text

hr

Password:

Plain Text

1234

(or whatever credentials you configured)

Otherwise, Postman may show:

Plain Text

401 Unauthorized

Check Application Startup

Before testing, verify you see something like this in the console:

Plain Text

Tomcat started on port 8080

Started EmployeeApplication

Then open:

Plain Text

http://localhost:8080/employees

in Postman.

If you share your current `EmployeeController`, `application.properties`, and Security configuration, I can verify the exact Postman URLs for your project.